

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Aplicaciones distribuidas

Generalidades de los Sistemas Distribuidos

Autor:

Cando Moreno Robinson Rodrigo, rcandom@uteq.edu.ec

Carrera:

Ingeniería de Software

Docente:

Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Quevedo, 2026

1 UNIDAD 1: Generalidades de los Sistemas Distribuidos

Los sistemas distribuidos agrupan nodos autónomos que cooperan mediante una red y entregan un servicio común aunque cada nodo conserva memoria, ejecución y fallos propios [1, 2]. La unidad aborda comunicación, coordinación, consistencia, disponibilidad y tolerancia a fallos porque esos conceptos explican el comportamiento de aplicaciones repartidas entre equipos, centros de datos, nube y borde [3, 1]. El estudio de estas generalidades resulta necesario porque las decisiones sobre transparencia, concurrencia y escalabilidad afectan la calidad del servicio cuando la red introduce latencia, particiones o fallos parciales [4, 5].

Los modelos de comunicación amplían la unidad porque ordenan la interacción entre clientes, servidores, pares, servicios híbridos y capas de soporte que conectan componentes distribuidos [2, 3]. La comunicación entre procesos aporta mecanismos concretos para mover solicitudes, respuestas y eventos entre nodos, mientras la sincronización define criterios de orden cuando cada máquina conserva su propio reloj [1, 4]. El análisis de modelos, middleware y relojes distribuidos complementa los fundamentos previos porque vincula la arquitectura con decisiones prácticas de despliegue, interoperabilidad y consistencia temporal [3, 5].

La seguridad y la confiabilidad amplían la unidad porque un sistema distribuido también debe proteger identidades, mensajes, datos, servicios y acuerdos operativos bajo condiciones de fallo o ataque [21, 28]. Las amenazas actuales afectan superficies repartidas entre microservicios, nube, borde, IoT y canales de comunicación, por lo que la arquitectura necesita controles de acceso, cifrado, monitoreo y alta disponibilidad desde el diseño [23, 29]. La relación entre protección y recuperación permite estudiar fallos, vulnerabilidades y continuidad del servicio como problemas conectados dentro de la misma infraestructura distribuida [24, 27].

La coordinación y la consistencia completan la unidad porque los nodos distribuidos necesitan acuerdos sobre orden, liderazgo, transacciones y estado compartido bajo latencia o particiones [4, 31]. Los modelos de consistencia, los algoritmos de elección, los protocolos de commit y los consensos replicados convierten operaciones concurrentes en resultados controlables para aplicaciones críticas [37, 32]. El estudio de CAP, Bully, Ring, 2PC, Paxos y Raft conecta teoría y práctica, debido a que cada mecanismo expresa límites distintos entre disponibilidad, seguridad, progreso y corrección [5, 41].

1.1 TEMA 1: Fundamentos de Sistemas Distribuidos

El tema precisa los principios que distinguen un sistema distribuido de una aplicación local, de una arquitectura centralizada y de un sistema paralelo orientado solo a acelerar una tarea computacional [1, 3]. Los fundamentos incluyen modelos de comunicación, replicación, ordenamiento de eventos, consistencia y recuperación, porque esos elementos determinan la interacción entre procesos concurrentes sin memoria global compartida [2, 4]. La revisión histórica y comparativa ayuda a relacionar servicios actuales de nube, microservicios e Internet de las Cosas con problemas persistentes de consenso, disponibilidad y tolerancia a fallos [6, 5].

1.1.1 Subtema 1: Definición, características y propiedades

Un sistema distribuido es una arquitectura formada por varios componentes autónomos que colaboran mediante mensajes para alcanzar un objetivo común y presentar una operación coordinada a usuarios o aplicaciones cliente [1, 3]. La autonomía de los nodos implica ausencia de memoria compartida, comunicación con latencia, conocimiento parcial del estado global y necesidad de protocolos para coordinar acciones concurrentes [2, 4]. Las características centrales comprenden comunicación entre servicios, descubrimiento de recursos, replicación, particionamiento, seguridad, observabilidad y recuperación, porque esos aspectos sostienen el funcionamiento de aplicaciones desplegadas en múltiples nodos [7, 9]. La Tabla 1 presenta esas características como una síntesis operativa para relacionar definición, diseño y comportamiento esperado del sistema [1, 2].

Tabla 1: Características principales de los sistemas distribuidos

Característica	Descripción
Comunicación por mensajes	Los nodos intercambian solicitudes, respuestas y eventos mediante protocolos de red, sin depender de memoria compartida.
Concurrencia	Varios componentes ejecutan operaciones al mismo tiempo y requieren coordinación para evitar estados incompatibles.
Escalabilidad	La arquitectura permite incorporar usuarios, datos o nodos mediante particionamiento, replicación y balanceo de carga.
Transparencia	El sistema oculta detalles de ubicación, acceso o replicación cuando esa ocultación no afecta el control de fallos.
Tolerancia a fallos	Los servicios continúan con operación total o degradada mediante detección, recuperación, redundancia y reasignación de recursos.
Heterogeneidad	Los componentes pueden usar distintas plataformas, redes o tecnologías, siempre que mantengan interfaces compatibles.

Las propiedades técnicas más relevantes son transparencia, apertura, escalabilidad, concurrencia, consistencia y tolerancia a fallos, ya que cada una describe una condición que la arquitectura debe conservar bajo operación distribuida [1, 3]. La transparencia oculta detalles de ubicación, acceso, migración o replicación cuando esa ocultación no elimina la necesidad de tratar latencia, errores de red y límites de disponibilidad [7, 8]. La escalabilidad combina particionamiento, replicación, balanceo y gestión de recursos, debido a que el aumento de usuarios, datos o dispositivos también incrementa comunicación, coordinación y consumo operativo [2, 9]. La Figura 1 muestra una representación general de clientes, red, servicios distribuidos y almacenamiento replicado como elementos conectados de una misma arquitectura [3, 2].

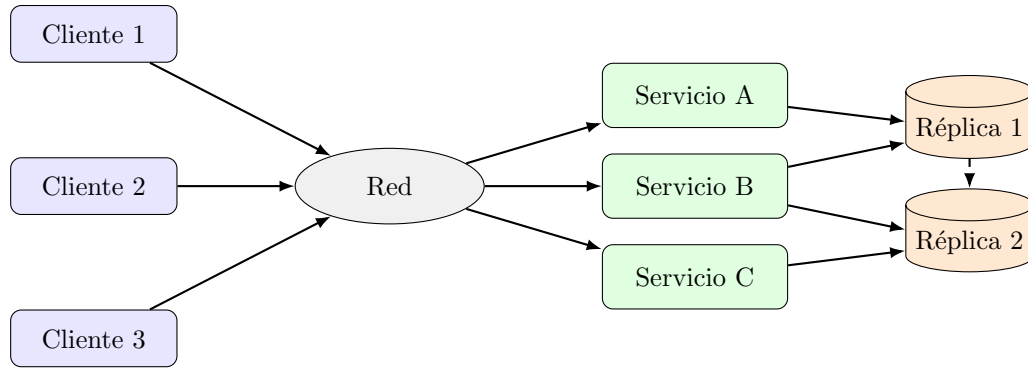


Figura 1: Representación general de un sistema distribuido

1.1.2 Subtema 2: Historia y evolución

La evolución de los sistemas distribuidos partió de redes y servicios cliente-servidor, pero el desarrollo reciente conecta esa base con nube, borde, IoT, cadenas de bloques, sistemas nativos de nube y análisis de datos distribuidos [1, 3]. Las arquitecturas modernas desplazaron muchas aplicaciones monolíticas hacia servicios pequeños que comunican por red, y esa transición volvió centrales las decisiones sobre despliegue, mensajería y límites entre componentes [7, 8]. Los sistemas intensivos en datos ampliaron la evolución hacia procesamiento repartido, flujos continuos y almacenamiento replicado, con retos de consistencia y disponibilidad ante fallos de máquinas o particiones de red [2, 5]. La Tabla 2 resume años, hitos y tecnologías que permiten ubicar esa evolución desde redes iniciales hasta arquitecturas de nube, borde y consenso distribuido [10, 3].

Tabla 2: Hitos y tecnologías relevantes en la evolución de los sistemas distribuidos

Año	Hito y tecnologías
1960–1970	Redes de computadoras, tiempo compartido y bases para comunicación entre máquinas autónomas.
1980	Modelos cliente-servidor, estaciones de trabajo conectadas y primeras abstracciones de llamada remota.
1990	Middleware, objetos distribuidos, CORBA, RMI y servicios empresariales con interfaces remotas.
2000	Computación en clúster, servicios web, almacenamiento distribuido y procesamiento masivo de datos.
2010	Computación en la nube, virtualización, contenedores, microservicios y despliegue elástico.
2020	Borde, IoT, blockchain, sistemas nativos de nube, consenso distribuido y análisis intensivo de datos.

Los problemas de consenso, detección de fallos y recuperación permanecen en la base histórica del área, aunque las revisiones recientes los conectan con entornos de nube, borde y gran escala [1, 6]. La disponibilidad y la consistencia evolucionaron hacia decisiones explícitas de diseño, porque los sistemas replicados deben responder bajo fallos sin perder criterios verificables de corrección [4, 5]. La evolución hacia microservicios e IoT incorporó más heterogeneidad, administración descentralizada y necesidades de confiabilidad, lo cual aumentó la importancia de patrones de comunicación

y despliegue [9, 7]. La Figura 2 muestra una línea de tiempo que relaciona las etapas principales con las tecnologías que marcaron la historia de los sistemas distribuidos [10, 3].

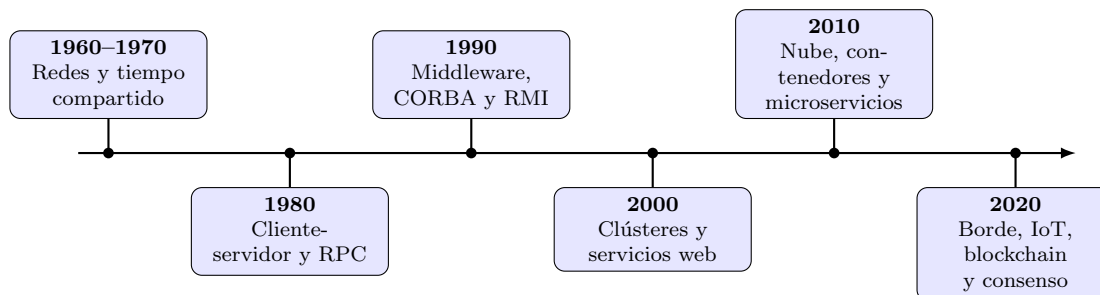


Figura 2: Línea de tiempo de la evolución de los sistemas distribuidos

1.1.3 Subtema 3: Ventajas, desafíos y retos

Las ventajas principales de un sistema distribuido incluyen compartición de recursos, crecimiento modular, disponibilidad mediante redundancia y cercanía del procesamiento respecto al usuario, al dato o al dispositivo de origen [2, 3]. La distribución permite ubicar componentes en varios nodos y aprovechar recursos heterogéneos, con lo cual una organización reduce dependencia de un único servidor y organiza servicios por responsabilidades técnicas [9, 7]. Los sistemas de análisis distribuido muestran que la división de datos y tareas mejora la capacidad de procesamiento, aunque esa mejora depende de coordinación eficiente y mecanismos claros de tolerancia a fallos [2, 6].

Los desafíos aparecen por concurrencia, comunicación no instantánea, ausencia de estado global completo y fallos parciales que afectan algunos nodos mientras otros continúan activos [1, 6]. La transparencia exige equilibrio técnico, porque ocultar por completo la distribución puede debilitar el tratamiento explícito de latencia, tiempo de espera, recuperación y límites de consistencia [3, 7]. La concurrencia demanda reglas de orden, coordinación y visibilidad, ya que operaciones simultáneas sobre réplicas o servicios pueden producir estados incompatibles si la arquitectura carece de criterios formales [4, 5].

La escalabilidad requiere particionar datos, replicar servicios y controlar cuellos de botella, porque el aumento de nodos o usuarios también incrementa tráfico, metadatos y coordinación entre componentes [2, 3]. La tolerancia a fallos necesita detección, aislamiento, recuperación y continuidad degradada, debido a que los fallos de proceso, red o almacenamiento pueden interrumpir una parte del sistema sin detener todo el servicio [1, 6]. El reto de consistencia y disponibilidad obliga a definir garantías según el contexto, porque una arquitectura replicada puede priorizar respuestas rápidas o estados más estrictos cuando aparecen particiones y fallos [4, 5].

1.1.4 Subtema 4: Comparación con sistemas centralizados y paralelos

Un sistema centralizado concentra estado, procesamiento y control en un servidor principal o en un núcleo de aplicación, por lo que su administración inicial resulta más directa mientras la de-

manda permanece dentro de una capacidad limitada [3, 8]. Ese enfoque facilita cambios locales y control de consistencia, aunque también concentra riesgo de interrupción y dificulta crecimiento cuando la carga exige particionamiento, despliegue independiente o comunicación entre servicios [7, 8]. Un sistema distribuido reparte componentes y responsabilidades entre nodos, por eso ofrece flexibilidad geográfica y organizacional a cambio de mayor complejidad en coordinación, monitoreo, recuperación y seguridad [1, 9].

Un sistema paralelo busca acelerar una computación mediante varios procesadores o núcleos que trabajan sobre una tarea común, mientras el sistema distribuido prioriza cooperación entre nodos autónomos conectados por red [3, 2]. La diferencia principal está en el objetivo operativo, porque el paralelismo enfatiza rendimiento de cálculo y el sistema distribuido enfatiza disponibilidad, recursos compartidos, heterogeneidad y continuidad del servicio [3, 1]. El grado de acoplamiento también cambia la comparación, ya que los sistemas paralelos suelen trabajar en entornos más controlados y los distribuidos deben tolerar latencia variable, fallos parciales y decisiones de consistencia [3, 4].

La arquitectura centralizada ofrece simplicidad inicial, pero su crecimiento exige máquinas más potentes o una migración posterior hacia particionamiento, microservicios y replicación [8, 7]. La arquitectura paralela ofrece alto rendimiento cuando una tarea admite división computacional clara, aunque la coordinación puede reducir el beneficio cuando el intercambio de datos supera el trabajo útil [3, 2]. La arquitectura distribuida resulta adecuada cuando el problema requiere compartir recursos, acercar servicios a usuarios, mantener disponibilidad o procesar datos ubicados en nodos distintos bajo decisiones explícitas sobre consistencia y fallos [4, 6]. La Figura 3 compara visualmente las diferencias principales entre sistemas centralizados, paralelos y distribuidos según control, objetivo y comunicación [3, 1].

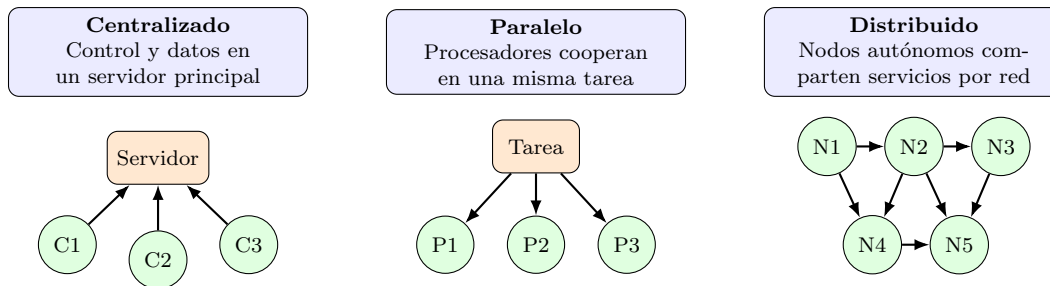


Figura 3: Comparación visual entre sistemas centralizados, paralelos y distribuidos

1.2 TEMA 2: Modelos y Comunicación en Sistemas Distribuidos

Los modelos y mecanismos de comunicación determinan cómo los componentes distribuidos intercambian solicitudes, respuestas, eventos y estado bajo condiciones de red variables [10, 11]. El tema analiza arquitecturas cliente-servidor, peer-to-peer e híbridas junto con sockets, RPC, RMI, middleware y sincronización lógica, porque esas piezas definen la relación entre diseño conceptual e implementación distribuida [12, 14]. La comunicación y el tiempo lógico requieren estudio conjunto, ya que un sistema distribuido necesita transportar mensajes y ordenar eventos sin depender de un reloj físico compartido entre todos los nodos [17, 18].

1.2.1 Subtema 1: Modelos de sistemas distribuidos

El modelo cliente-servidor organiza el sistema alrededor de clientes que solicitan recursos y servidores que concentran funciones de procesamiento, datos o coordinación [10, 7]. Esta organización simplifica control, seguridad y administración del servicio, aunque el crecimiento de usuarios exige balanceo, replicación y despliegue cuidadoso para evitar cuellos de botella [8, 16]. Las aplicaciones web, móviles y empresariales conservan este modelo cuando un componente central expone interfaces estables y los clientes consumen esas interfaces mediante protocolos de red [7, 14]. La Figura 4 representa visualmente la relación entre clientes, servidor y almacenamiento central en este modelo [10, 7].

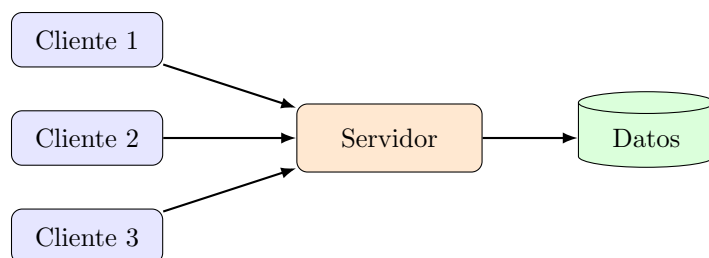


Figura 4: Modelo cliente-servidor

El modelo peer-to-peer distribuye responsabilidades entre nodos que pueden actuar como consumidores y proveedores de recursos dentro de la misma red lógica [12, 13]. Esta forma reduce dependencia de un servidor único y favorece compartición directa de almacenamiento, contenido o cómputo, aunque exige mecanismos de descubrimiento, incentivos, reputación y control de disponibilidad [12, 13]. Los modelos híbridos combinan coordinación central con cooperación entre nodos, y esa mezcla aparece en arquitecturas de borde, nube e IoT que reparten trabajo según latencia, capacidad y ubicación de datos [11, 16]. La Figura 5 contrasta el intercambio directo de un modelo peer-to-peer con la coordinación parcial que caracteriza a un modelo híbrido [12, 16].

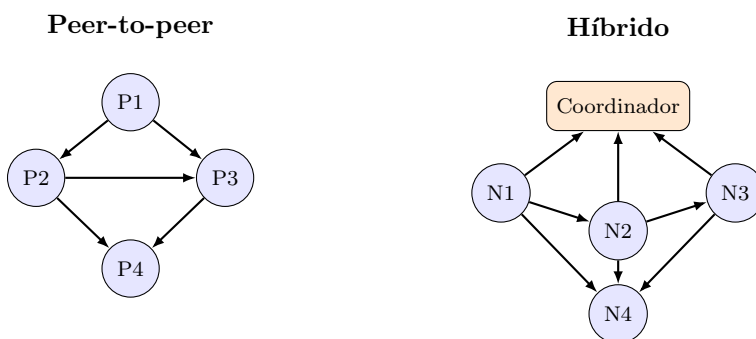


Figura 5: Modelos peer-to-peer e híbrido

1.2.2 Subtema 2: Comunicación entre procesos: sockets, RPC, RMI

Los sockets ofrecen una interfaz de bajo nivel para que procesos intercambien bytes sobre protocolos de transporte, por lo cual resultan útiles cuando la aplicación requiere control explícito de conexiones, puertos, mensajes y manejo de errores [10, 15]. Este mecanismo entrega flexibilidad, pero traslada al desarrollador responsabilidades de serialización, delimitación de mensajes, reintentos, seguridad y administración de fallos de red [15, 14]. Los sistemas basados en microservicios usan sockets de forma directa o indirecta cuando bibliotecas, proxies o mallas de servicio gestionan conexiones entre procesos distribuidos [7, 15]. La Figura 6 compara el nivel de abstracción entre sockets, RPC y RMI para mostrar cómo cada mecanismo desplaza responsabilidades entre la aplicación, el transporte y la invocación remota [10, 14].

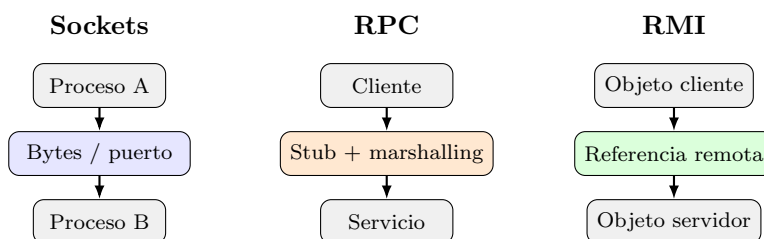


Figura 6: Niveles de abstracción en la comunicación entre procesos

La llamada a procedimiento remoto permite invocar una operación ubicada en otro proceso con una abstracción cercana a una llamada local, aunque la red añade latencia, fallos y costos de serialización que la interfaz debe manejar [10, 14]. RPC es común en servicios modernos porque estructura contratos de operación, formatos de mensajes y respuestas, mientras RMI extiende esa idea hacia invocación de métodos sobre objetos remotos [10, 14]. La diferencia práctica frente a sockets está en el nivel de abstracción, ya que sockets exponen comunicación básica y RPC o RMI encapsulan transporte, marshalling, nombres de operación y manejo de respuestas [14, 7].

1.2.3 Subtema 3: Middleware y su rol en la distribución

El middleware es una capa de software que ubica servicios comunes entre aplicaciones y sistemas operativos para facilitar comunicación, descubrimiento, seguridad, observabilidad y administración de componentes distribuidos [10, 15]. Esta capa reduce acoplamiento entre clientes y servicios porque concentra funciones transversales como balanceo, autenticación, monitoreo, enrutamiento y control de políticas [15, 7]. Las arquitecturas de microservicios muestran esa función con gateways, brokers, registros de servicios y mallas de servicio que coordinan tráfico sin trasladar toda la complejidad a cada aplicación [7, 15]. La Tabla 3 resume funciones frecuentes de middleware y su aporte dentro de una arquitectura distribuida [10, 15].

Tabla 3: Funciones del middleware en sistemas distribuidos

Función	Aporte a la distribución
Comunicación	Encapsula protocolos, formatos de mensajes y canales para que los servicios interactúen con menor acoplamiento.
Descubrimiento	Permite localizar servicios disponibles y actualizar rutas cuando cambian instancias, réplicas o ubicaciones.
Seguridad	Centraliza autenticación, autorización, cifrado y políticas de acceso entre componentes distribuidos.
Observabilidad	Reúne métricas, trazas y registros que facilitan diagnóstico de latencia, errores y dependencias.
Enrutamiento	Distribuye solicitudes mediante balanceo, reglas de tráfico, reintentos y control de versiones.
Interoperabilidad	Oculta diferencias de plataforma, lenguaje o infraestructura mediante interfaces y adaptadores comunes.

El middleware también facilita interoperabilidad entre plataformas heterogéneas, debido a que encapsula detalles de protocolos, formatos de datos y ubicación de recursos en redes con nodos diversos [10, 16]. En entornos de nube, borde e IoT, esta capa ayuda a distribuir procesamiento cerca del dato y a coordinar servicios ubicados en infraestructuras con capacidades distintas [11, 16]. El rol del middleware exige límites claros porque una capa demasiado pesada puede añadir latencia, dependencia operativa y puntos de congestión dentro del sistema [15, 11].

1.2.4 Subtema 4: Sincronización y relojes distribuidos

La sincronización en sistemas distribuidos enfrenta la ausencia de un reloj global confiable, por lo que el orden de los eventos depende de relaciones causales observables mediante envío y recepción de mensajes [17, 18]. El algoritmo de Lamport asigna marcas lógicas crecientes a eventos locales y mensajes, y esa regla permite inferir que un evento ocurrió antes que otro cuando existe una cadena causal entre ambos [17, 18]. Esta técnica no mide tiempo físico, pero aporta una base para ordenar eventos, depurar ejecuciones, coordinar protocolos y razonar sobre consistencia en ausencia de sincronía perfecta [20, 4].

Los relojes físicos complementan los relojes lógicos cuando una aplicación requiere límites temporales reales, mediciones de latencia o coordinación con dispositivos externos [19, 16]. Las arquitecturas distribuidas modernas combinan sincronización física, tiempo lógico y verificación de propiedades temporales para controlar eventos concurrentes en servicios, sensores y flujos de datos [20, 19]. La elección entre reloj físico, reloj lógico o una combinación depende de los requisitos de causalidad, precisión, costo de comunicación y tolerancia a fallos de la aplicación distribuida [17, 20].

1.3 TEMA 3: Seguridad y Confiabilidad en Sistemas Distribuidos

La seguridad y la confiabilidad definen la capacidad de un sistema distribuido para resistir ataques, limitar accesos indebidos, proteger información y continuar operando bajo fallos parciales [21, 1].

El tema estudia amenazas, autenticación, autorización, cifrado, detección de fallos, recuperación, SLA y alta disponibilidad porque esos componentes sostienen la confianza operativa de servicios repartidos en múltiples nodos [23, 28]. La integración de controles preventivos y mecanismos de recuperación resulta necesaria porque un incidente de seguridad también puede convertirse en degradación, indisponibilidad o incumplimiento de acuerdos de servicio [24, 29].

1.3.1 Subtema 1: Amenazas y vulnerabilidades en entornos distribuidos

Las amenazas en entornos distribuidos incluyen interceptación de mensajes, credenciales expuestas, configuraciones débiles, abuso de interfaces, movimiento lateral y fallos de aislamiento entre servicios [21, 22]. La distribución aumenta la superficie de ataque porque cada API, canal, nodo, contenedor, servicio o componente de borde añade puntos de entrada y dependencias operativas [23, 24]. Las vulnerabilidades de microservicios muestran que errores de autenticación, autorización, comunicación y gestión de secretos pueden propagarse entre componentes conectados por red [21, 15].

La Tabla 4 organiza amenazas y vulnerabilidades frecuentes en entornos distribuidos junto con el daño que pueden ocasionar sobre confidencialidad, integridad, disponibilidad y trazabilidad del servicio [21, 22, 23].

Tabla 4: Amenazas y vulnerabilidades comunes en entornos distribuidos

Amenaza o vulnerabilidad	Causa frecuente	Daño posible
Interceptación de mensajes	Canales sin cifrado, certificados mal gestionados o tráfico expuesto entre servicios.	Robo de credenciales, fuga de datos sensibles y pérdida de confidencialidad en tránsito.
Credenciales expuestas	Secretos en código, tokens sin rotación o permisos compartidos entre componentes.	Acceso no autorizado, escalamiento de privilegios y abuso de servicios internos.
Configuraciones débiles	Puertos abiertos, políticas por defecto, reglas amplias o servicios publicados sin control.	Mayor superficie de ataque, intrusiones externas y ejecución de operaciones no previstas.
Abuso de API	Falta de validación, límites de uso insuficientes o autenticación incompleta.	Manipulación de datos, consumo excesivo de recursos e interrupción parcial del servicio.
Movimiento lateral	Segmentación deficiente, confianza excesiva entre nodos o permisos internos amplios.	Propagación del compromiso desde un servicio vulnerable hacia otros componentes críticos.
Fallo de aislamiento	Contenedores, máquinas virtuales o microservicios con límites de seguridad débiles.	Exposición de información entre componentes y afectación simultánea de varios servicios.
Gestión deficiente de dependencias	Bibliotecas obsoletas, imágenes vulnerables o componentes externos sin revisión.	Explotación de vulnerabilidades conocidas, pérdida de integridad y fallos operativos.

El riesgo también aparece cuando una arquitectura mezcla nubes, IoT, cadenas de bloques, SDN

o servicios externos con políticas de seguridad inconsistentes [23, 16]. Los roles de seguridad en microservicios identifican decisiones de diseño que debilitan confidencialidad, integridad, disponibilidad o trazabilidad antes de que el sistema llegue a producción [22, 21]. La mitigación exige inventario de activos, límites de confianza, segmentación, observabilidad y revisión continua de dependencias, debido a que la amenaza cambia con cada servicio añadido o reconfigurado [24, 23].

1.3.2 Subtema 2: Autenticación, autorización y cifrado

La autenticación confirma la identidad de usuarios, servicios o dispositivos antes de permitir interacción con recursos distribuidos, y esa validación necesita credenciales, certificados, tokens o mecanismos equivalentes con ciclo de vida controlado [23, 24]. La autorización determina qué operaciones permite cada identidad autenticada, por lo que los modelos de control de acceso deben expresar permisos granulares y revisables entre aplicaciones, datos y servicios [23, 25]. El cifrado protege datos en tránsito y en reposo, y las técnicas basadas en atributos aportan control criptográfico cuando múltiples entidades acceden a información bajo políticas diferentes [25, 24].

El enfoque de confianza cero fortalece sistemas distribuidos porque trata cada solicitud como una interacción que necesita verificación explícita, contexto y autorización mínima [24, 23]. En microservicios, los controles de identidad entre servicios reducen exposición cuando un componente comprometido intenta invocar operaciones fuera de su función autorizada [21, 22]. La protección efectiva combina autenticación fuerte, autorización por política, cifrado, rotación de secretos y monitoreo, debido a que un solo control no cubre todas las rutas de ataque de una arquitectura distribuida [25, 24].

1.3.3 Subtema 3: Tolerancia a fallos: detección y recuperación

La tolerancia a fallos permite que un sistema distribuido continúe con servicio total o degradado cuando algunos nodos, enlaces, procesos o almacenamientos fallan [1, 27]. La detección de fallos usa latidos, tiempos de espera, observación de métricas, consenso o verificaciones cruzadas para distinguir retrasos transitorios de interrupciones reales [1, 6]. La recuperación restaura capacidad mediante reinicio, conmutación por error, replicación, reintentos controlados, reconstrucción de estado o reasignación de tareas hacia recursos disponibles [6, 26].

Los sistemas de análisis distribuido requieren tolerancia a fallos porque una tarea larga puede perder progreso cuando fallan nodos de cómputo, almacenamiento o comunicación [6, 27]. La replicación de solicitudes en plataformas FaaS reduce el impacto de respuestas lentas o fallidas, aunque también incrementa consumo de recursos y exige control de duplicados [26, 29]. La recuperación confiable necesita políticas explícitas de degradación, prioridades de servicio y verificación posterior, porque la continuidad sin consistencia puede dañar datos o producir resultados incorrectos [5, 27].

1.3.4 Subtema 4: Acuerdos de nivel de servicio y alta disponibilidad

Un acuerdo de nivel de servicio define compromisos medibles sobre disponibilidad, rendimiento, tiempos de respuesta, soporte, seguridad u otros indicadores que el proveedor debe cumplir [28, 30]. La alta disponibilidad busca mantener el servicio accesible mediante redundancia, balanceo, conmutación por error, monitoreo y ubicación adecuada de recursos [29, 26]. La relación entre SLA y disponibilidad convierte métricas técnicas en obligaciones operativas, porque una caída, degradación o latencia excesiva puede afectar cumplimiento contractual y calidad percibida [28, 29].

El monitoreo de SLA necesita recopilar métricas en tiempo real, comparar umbrales y registrar evidencias de cumplimiento o violación durante la operación del sistema [30, 28]. La ubicación de máquinas virtuales y réplicas influye en disponibilidad porque fallos correlacionados de infraestructura pueden afectar varios componentes si todos dependen de los mismos dominios físicos o lógicos [29, 26]. La estrategia de alta disponibilidad debe equilibrar costo, redundancia, consistencia y tiempo de recuperación, debido a que mayor resiliencia también consume recursos y añade coordinación entre nodos [6, 28].

1.4 TEMA 4: Coordinación y Consistencia Distribuida

La coordinación distribuida reúne mecanismos que permiten elegir responsables, ordenar acciones, confirmar operaciones y mantener réplicas coherentes cuando varios nodos ejecutan tareas concurrentes [34, 37]. El tema analiza CAP, elección de líder, commit en dos fases, Paxos y Raft porque esos conceptos explican cómo una arquitectura decide entre progreso, corrección y disponibilidad bajo fallos [4, 32]. La consistencia distribuida requiere protocolos explícitos, ya que la replicación y el paralelismo aumentan rendimiento y disponibilidad pero también introducen conflictos de estado, mensajes retrasados y decisiones parciales [2, 5].

1.4.1 Subtema 1: El problema de la consistencia y el modelo CAP

El problema de la consistencia aparece cuando varias réplicas aceptan lecturas y escrituras concurrentes y la aplicación necesita definir qué valor resulta válido ante retrasos o particiones [4, 5]. El modelo CAP organiza la discusión alrededor de consistencia, disponibilidad y tolerancia a particiones, y las revisiones recientes mantienen ese marco como una guía para decisiones de diseño en servicios replicados [4, 31]. La partición de red obliga a priorizar respuestas disponibles o estados más estrictos, debido a que la comunicación incompleta limita la capacidad de confirmar una única visión global del dato [5, 2].

La consistencia fuerte exige que las operaciones observables respeten un orden compatible con una única historia lógica, aunque ese objetivo incrementa coordinación y latencia bajo fallos [4, 5]. La consistencia eventual acepta divergencias temporales y busca convergencia posterior, por lo que resulta útil cuando la disponibilidad pesa más que la lectura inmediata del último valor confirmado [2, 31]. La selección de un modelo de consistencia depende de la semántica del dato, el costo de coordinación, el impacto del error y los acuerdos de disponibilidad de la aplicación [28, 29].

1.4.2 Subtema 2: Algoritmos de coordinación y elección de líder

La elección de líder asigna a un nodo la responsabilidad de coordinar acciones compartidas, ordenar solicitudes o activar recuperación cuando el coordinador anterior falla [34, 35]. Un algoritmo de elección debe garantizar seguridad, porque dos líderes simultáneos pueden emitir decisiones incompatibles, y también debe garantizar progreso cuando la red entrega mensajes suficientes [36, 34]. Los diseños recientes evalúan carga, tasa de fallos, anonimato o tamaño de grupo, lo cual muestra que la elección de líder depende del modelo de red y de los supuestos de fallo [35, 36].

El algoritmo Bully prioriza identificadores o prioridades altas para seleccionar coordinador, por eso resulta simple en grupos con membresía conocida y comunicación directa entre nodos [36, 35]. El algoritmo Ring organiza los nodos en un anillo lógico y transmite la elección por sucesores hasta identificar al candidato con mayor prioridad dentro de la ronda [35, 34]. Ambos esquemas ilustran decisiones básicas de coordinación, aunque sistemas actuales suelen añadir detección de fallos, control de carga y tolerancia a mensajes perdidos [1, 6].

1.4.3 Subtema 3: Transacciones distribuidas y commit en dos fases

Las transacciones distribuidas agrupan operaciones ubicadas en varios nodos y necesitan atomicidad para que todos los participantes confirmen o todos descarten el cambio [37, 38]. El commit en dos fases divide la decisión en una etapa de preparación y otra de confirmación, donde el coordinador recopila votos y luego comunica commit o abort según las respuestas recibidas [37, 38]. Este protocolo ofrece una estructura comprensible para atomicidad distribuida, aunque el bloqueo puede aparecer cuando participantes esperan una decisión del coordinador bajo fallos o particiones [5, 37].

Las variantes recientes de commit distribuido exploran mejoras de vivacidad, tolerancia a fallos y latencia sin abandonar la necesidad de una decisión común entre participantes [38, 37]. La integración con consenso o replicación aumenta resiliencia, pero también añade mensajes, estados persistentes y reglas de recuperación que el sistema debe administrar con precisión [39, 40]. La elección entre 2PC, variantes tolerantes a fallos o consenso completo depende del costo aceptable de bloqueo, la criticidad de los datos y el nivel de disponibilidad requerido [28, 29].

1.4.4 Subtema 4: Sistemas de consenso: Paxos y Raft

Los sistemas de consenso permiten que varios nodos acuerden un valor o una secuencia de decisiones incluso cuando algunos procesos fallan o la red retrasa mensajes [32, 31]. Paxos basa su seguridad en quórum superpuestos y roles lógicos que evitan decisiones contradictorias cuando diferentes propuestas compiten dentro del sistema [40, 39]. Raft organiza el consenso alrededor de elección de líder, replicación de bitácora y compromiso de entradas, lo cual facilita la explicación operativa del protocolo en sistemas replicados [41, 32].

Paxos y Raft comparten el objetivo de mantener réplicas coherentes, pero difieren en la forma de estructurar liderazgo, bitácoras y recuperación ante cambios de membresía o fallos [41, 31]. Las revisiones recientes de consenso ubican estos protocolos junto a variantes usadas en bases de datos,

cadena de bloques y servicios de infraestructura que necesitan decisiones verificables entre nodos [32, 33]. La adopción de consenso exige evaluar latencia, número de mensajes, tolerancia a fallos y complejidad de implementación, porque la corrección formal pierde valor si el costo operativo supera las necesidades del servicio [40, 5].

Referencias

- [1] B. Chaurasia, A. Verma, and P. Verma, “An in-depth and insightful exploration of failure detection in distributed systems,” *Computer Networks*, vol. 247, Art. no. 110432, 2024. doi: <https://doi.org/10.1016/j.comnet.2024.110432>
- [2] A. Margara, G. Cugola, N. Felicioni, and S. Cilloni, “A model and survey of distributed data-intensive systems,” *ACM Computing Surveys*, vol. 56, no. 1, Art. no. 16, pp. 1–69, 2024. doi: <https://doi.org/10.1145/3604801>
- [3] F. Dai, M. A. Hossain, and Y. Wang, “State of the art in parallel and distributed systems: Emerging trends and challenges,” *Electronics*, vol. 14, no. 4, Art. no. 677, 2025. doi: <https://doi.org/10.3390/electronics14040677>
- [4] E. A. Lee, S. Bateni, S. Lin, M. Lohstroh, and C. Menard, “Trading off consistency and availability in tiered heterogeneous distributed systems,” *Intelligent Computing*, vol. 2, Art. no. 0013, 2023. doi: <https://doi.org/10.34133/icomputing.0013>
- [5] M. Kokocinski, T. Kobus, and P. T. Wojciechowski, “On the correctness of highly available systems in the presence of failures,” *Journal of Parallel and Distributed Computing*, vol. 180, Art. no. 104707, 2023. doi: <https://doi.org/10.1016/j.jpdc.2023.04.008>
- [6] S. Isukapalli and S. N. Srirama, “A systematic survey on fault-tolerant solutions for distributed data analytics: Taxonomy, comparison, and future directions,” *Computer Science Review*, vol. 53, Art. no. 100660, 2024. doi: <https://doi.org/10.1016/j.cosrev.2024.100660>
- [7] I. K. Aksakalli, T. Celik, A. B. Can, and B. Tekinerdogan, “Deployment and communication patterns in microservice architectures: A systematic literature review,” *Journal of Systems and Software*, vol. 180, Art. no. 111014, 2021. doi: <https://doi.org/10.1016/j.jss.2021.111014>
- [8] Y. M. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. F. Buckley, and P. Clarke, “Decomposition of monolith applications into microservices architectures: A systematic review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023. doi: <https://doi.org/10.1109/TSE.2023.3287297>
- [9] H. Siddiqui, F. Khendek, and M. Toeroe, “Microservices based architectures for IoT systems: State-of-the-art review,” *Internet of Things*, vol. 23, Art. no. 100854, 2023. doi: <https://doi.org/10.1016/j.iot.2023.100854>
- [10] C. Ramón-Cortés, P. Álvarez, F. Lordan, J. Álvarez, J. Ejarque, and R. M. Badia, “A survey on the distributed computing stack,” *Computer Science Review*, vol. 42, Art. no. 100422, 2021. doi: <https://doi.org/10.1016/j.cosrev.2021.100422>

- [11] D. Rosendo, A. Costan, P. Valduriez, and G. Antoniu, “Distributed intelligence on the edge-to-cloud continuum: A systematic literature review,” *Journal of Parallel and Distributed Computing*, vol. 166, pp. 71–94, 2022. doi: <https://doi.org/10.1016/j.jpdc.2022.04.004>
- [12] C. Ihle, D. Trautwein, M. Schubotz, N. Meuschke, and B. Gipp, “Incentive mechanisms in peer-to-peer networks: A systematic literature review,” *ACM Computing Surveys*, vol. 55, no. 14s, Art. no. 308, pp. 1–69, 2023. doi: <https://doi.org/10.1145/3578581>
- [13] E. Daniel and F. Tschorsch, “IPFS and friends: A qualitative comparison of next generation peer-to-peer data networks,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 1, pp. 31–52, 2022. doi: <https://doi.org/10.1109/COMST.2022.3143147>
- [14] L. Zhang, K. Pang, J. Xu, and B. Niu, “High performance microservice communication technology based on modified remote procedure call,” *Scientific Reports*, vol. 13, Art. no. 12141, 2023. doi: <https://doi.org/10.1038/s41598-023-39355-4>
- [15] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, “Dissecting overheads of service mesh sidecars,” in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 2023, pp. 142–157. doi: <https://doi.org/10.1145/3620678.3624652>
- [16] A. Dauda, O. Flauzac, and F. Nolot, “A survey on IoT application architectures: From cloud to edge and fog computing,” *Sensors*, vol. 24, no. 16, Art. no. 5320, 2024. doi: <https://doi.org/10.3390/s24165320>
- [17] M. Acclavio, R. Horne, S. Mauw, and L. Straßburger, “A graphical proof theory of logical time,” in *7th International Conference on Formal Structures for Computation and Deduction*, vol. 228, 2022, pp. 22:1–22:25. doi: <https://doi.org/10.4230/LIPIcs.FSCD.2022.22>
- [18] S. Sokoto and E. Onur, “SGLC: A logical clock using succinct graphs,” in *2022 30th Signal Processing and Communications Applications Conference*, 2022, pp. 1–4. doi: <https://doi.org/10.1109/SIU55565.2022.9864718>
- [19] D. Chefrour, “Evolution of network time synchronization towards nanoseconds accuracy: A survey,” *Computer Communications*, vol. 191, pp. 26–35, 2022. doi: <https://doi.org/10.1016/j.comcom.2022.04.023>
- [20] R. Ganguly, Y. Xue, A. Jonckheere, P. Ljung, B. Schornstein, B. Bonakdarpour, and M. Herlihy, “Distributed runtime verification of metric temporal properties,” *Journal of Parallel and Distributed Computing*, vol. 185, Art. no. 104801, 2024. doi: <https://doi.org/10.1016/j.jpdc.2023.104801>
- [21] P. Haindl, P. Kochberger, and A. Sveggen, “A systematic literature review of inter-service security threats and mitigation strategies in microservice architectures,” *IEEE Access*, vol. 12, pp. 77316–77345, 2024. doi: <https://doi.org/10.1109/ACCESS.2024.3406500>
- [22] F. Ponce, J. Soldani, H. Astudillo, and A. Brogi, “Smells and refactorings for microservices security: A multivocal literature review,” *Journal of Systems and Software*, vol. 192, Art. no. 111393, 2022. doi: <https://doi.org/10.1016/j.jss.2022.111393>
- [23] L. Golightly, V. Chang, Q. A. Xu, X. Gao, and B. S. Liu, “Securing distributed systems: A survey on access control techniques for cloud, blockchain, IoT and SDN,” *Cyber Security*

- and Applications*, vol. 1, Art. no. 100015, 2023. doi: <https://doi.org/10.1016/j.csa.2023.100015>
- [24] H. Kang, J. Liu, D. Li, and B. Li, “Theory and application of zero trust security: A brief survey,” *Entropy*, vol. 25, no. 12, Art. no. 1595, 2023. doi: <https://doi.org/10.3390/e25121595>
 - [25] M. Rasori, P. Perazzo, and G. Dini, “A survey on attribute-based encryption schemes suitable for the Internet of Things,” *IEEE Internet of Things Journal*, vol. 9, no. 11, pp. 8269–8290, 2022. doi: <https://doi.org/10.1109/JIOT.2022.3154039>
 - [26] M. Bouizem, N. Benameur, A. Carpen-Amarie, and A. Costan, “Integrating request replication into FaaS platforms: An experimental evaluation,” *Journal of Cloud Computing*, vol. 12, Art. no. 64, 2023. doi: <https://doi.org/10.1186/s13677-023-00457-z>
 - [27] D. S. de Souza, C. E. da Silva, C. S. Z. de Souza, G. Vale, and R. de S. Torres, “A systematic review of fault tolerance techniques for smart city applications,” *Journal of Systems and Software*, vol. 219, Art. no. 112249, 2025. doi: <https://doi.org/10.1016/j.jss.2024.112249>
 - [28] S. Qazi, B. A. Khawaja, and Q. U. Farooq, “Service level agreement in cloud computing: Taxonomy, prospects, and challenges,” *Internet of Things*, vol. 26, Art. no. 101126, 2024. doi: <https://doi.org/10.1016/j.iot.2024.101126>
 - [29] J. Liu, Z. Ye, J. Zhang, J. Huang, and J. Li, “Toward a service availability-guaranteed cloud through VM placement optimization,” *IEEE/ACM Transactions on Networking*, vol. 32, no. 6, pp. 4800–4813, 2024. doi: <https://doi.org/10.1109/TNET.2024.3401758>
 - [30] A. Badshah, A. Jalal, U. Farooq, G.-U. Rehman, S. S. Band, and C. Iwendi, “SLA monitoring as a service: A big data infrastructure for a monitoring cloud,” *Big Data*, vol. 11, no. 5, pp. 339–354, 2023. doi: <https://doi.org/10.1089/big.2021.0274>
 - [31] D. T. Dang and D. Hwang, “Consensus-based methods for distributed systems, blockchain, and voting: A survey,” *Journal of Information and Telecommunication*, vol. 9, no. 2, pp. 237–260, 2025. doi: <https://doi.org/10.1080/24751839.2024.2416728>
 - [32] S. Zhou, K. Li, L. Xiao, J. Cai, W. Liang, and A. Castiglione, “A systematic review of consensus mechanisms in blockchain,” *Mathematics*, vol. 11, no. 10, Art. no. 2248, 2023. doi: <https://doi.org/10.3390/math11102248>
 - [33] H. Kim and D. Kim, “A taxonomic hierarchy of blockchain consensus algorithms: An evolutionary phylogeny approach,” *Sensors*, vol. 23, no. 5, Art. no. 2739, 2023. doi: <https://doi.org/10.3390/s23052739>
 - [34] B. Gorain, A. Miller, and A. Pelc, “Four shades of deterministic leader election in anonymous networks,” *Distributed Computing*, vol. 36, pp. 419–449, 2023. doi: <https://doi.org/10.1007/s00446-023-00451-3>
 - [35] A. Biswas, A. K. Maurya, A. K. Tripathi, and S. Aknine, “FRLLE: A failure rate and load-based leader election algorithm for a bidirectional ring in distributed systems,” *The Journal of Supercomputing*, vol. 77, no. 1, pp. 751–779, 2021. doi: <https://doi.org/10.1007/s11227-020-03286-y>

- [36] S. Srinivasan and R. Kandukoori, “A synod based deterministic and indulgent leader election protocol for asynchronous large groups,” *International Journal of Parallel, Emergent and Distributed Systems*, vol. 37, no. 2, pp. 220–247, 2022. doi: <https://doi.org/10.1080/17445760.2021.1879067>
- [37] G. Chockler and A. Gotsman, “Multi-shot distributed transaction commit,” *Distributed Computing*, vol. 34, no. 4, pp. 301–318, 2021. doi: <https://doi.org/10.1007/s00446-021-00389-4>
- [38] C. Jensen, H. Howard, A. Katsarakis, and R. Mortier, “Unanimous 2PC: Fault-tolerant distributed transactions can be fast and simple,” in *Proceedings of the 11th Workshop on Principles and Practice of Consistency for Distributed Data*, 2024, pp. 44–57. doi: <https://doi.org/10.1145/3642976.3653035>
- [39] K. Antoniadis, J. Benhaim, A. Desjardins, E. Poroma, V. Gramoli, R. Guerraoui, G. Voron, and I. Zablotchi, “Leaderless consensus,” *Journal of Parallel and Distributed Computing*, vol. 176, pp. 95–113, 2023. doi: <https://doi.org/10.1016/j.jpdc.2023.01.009>
- [40] C. Mao, W. Golab, and B. Wong, “Antipaxos: Taking interactive consistency to the next level,” *Journal of Parallel and Distributed Computing*, vol. 187, Art. no. 104839, 2024. doi: <https://doi.org/10.1016/j.jpdc.2024.104839>
- [41] P. Bora, P. D. Minh, and T. A. C. Willemse, “Modelling the Raft distributed consensus protocol in mCRL2,” *Electronic Proceedings in Theoretical Computer Science*, vol. 399, pp. 7–20, 2024. doi: <https://doi.org/10.4204/EPTCS.399.4>